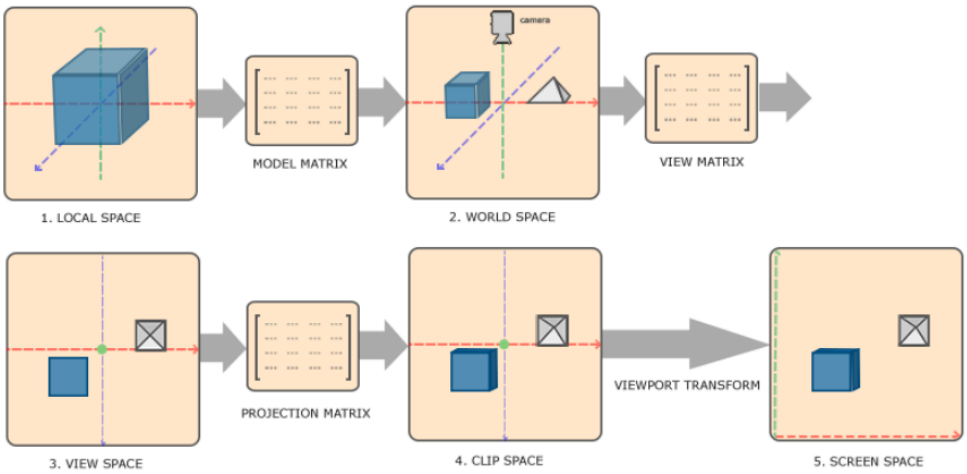
T4. Visión, 2D-3D

View Space y Clip Space

El **View Space** es el sistema de coordenadas de cámara o de punto de vista del observador. La **View Matrix** transforma las coordenadas de escena a coordenadas de cámara mediante una combinación de traslaciones y rotaciones. La cámara se ubica siempre en el $(0,\ 0,\ 0)$ mirando hacia el eje $-\ Z$. Para conseguir esto, se deberían realizar ciertas **transformaciones** sobre la **posición** de la cámara en el world space. En su lugar, a la escena se le aplica la **inversa de las transformaciones** que se le aplicarían a la cámara.

El **Clip Space**, una vez obtenidas las coordenadas en el espacio de vista (view space), se realiza un proceso de **recorte** y **proyección** (clipping) para descartar los elementos que se encuentran fuera del volumen definido por el **frustum** (volumen de visión de la cámara, área del espacio que la cámara verá). Los vértices que quedan fuera de este volumen son **eliminados**, mientras que los restantes se procesan para su posterior conversión en fragmentos. Por defecto, el frustrum es $[-\ 1, 1]$ en todos los ejes (este rango corresponde al espacio de coordenadas normalizadas).

La **Projection Matrix** permite cambiar las **dimensiones** del frustrum. Después de aplicar la matriz de proyección se realiza una **división perspectiva** que convierte las coordenadas del clip space a **NCD** (normalized device coordinates), diviendo cada componente $x,\ y,\ z$ por el componente homogéneo w.



Proyección

Una **proyección** es una **transformación** que reduce la dimensionalidad de una espacio, normalmente de R^3 a R^2. Es decir, es el proceso de convertir una escena tridimensional en una imagen bidimensional.

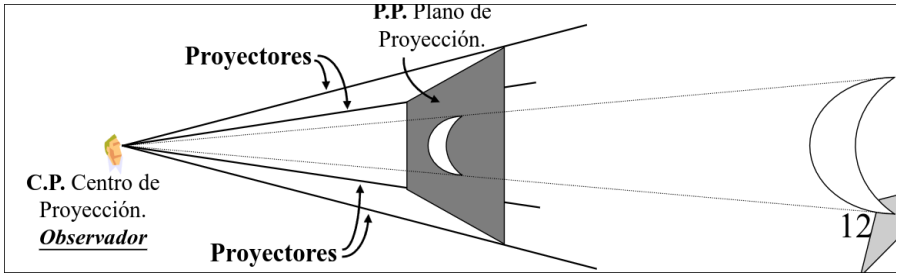
- <u>Elementos de una proyección</u>

- **Objetos a proyectar** (escena): elementos tridimensionales que componen el mundo virtual.

- **Plano de proyección**: superficie 2D en la que se representan los objetos 3D. En la práctica se corresponde a la **pantalla** o **viewport**.

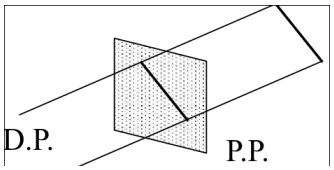
- **Punto de vista** o **centro de proyección**: posición desde donde se observa la escena. En la práctica se corresponde a la **ubicación** de la **cámara.**

- **Proyectores**: rayos imaginarios que se trazan desde los puntos del objeto hasta el centro de proyección (CP). Los puntos en los que intersecan con el plano de proyección (PP) determinan dónde de dibujará cada punto del objeto en la imagen final.

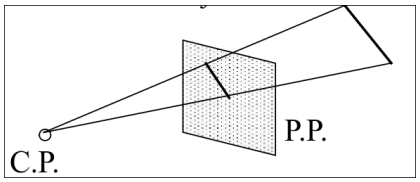


- <u>Tipos de Proyecciones</u>

- **Proyección ortográfica** o **paralela**, se caracteriza porque el **centro de proyección** está situado en el **infinito**, por lo que los proyectores son paralelos entre sí. Se define una dirección de proyeción en vez de un centro de proyacción.



- **Proyección perspectiva**, se caracteriza porque el **centro de proyección** está situado en el **espacio**, por lo que los proyectores forman un **cono** cuyo vértice es el centro de proyacción. Proporciona **realismo visual** y da sensación tridimensional. SIn embargo, no mantiene la forma real de los objetos ni la escala.



- **Proyección ortográfica multivista**. El **plano de proyección** es **perpendicular** a algún eje, por lo que se necesitan **varias vistas** para obtener una vision 3D de los objetos.
    - **Planta**: el plano de proyección es perpendicular al eje Y.
    - **Alzado**: el plano de proyección es perpendicular al eje Z.
    - **Perfil**: el plano de proyección es perpendicular al eje X.

  Es posible realizar **medidas precisas** y todas las vistas tienen la **misma escala**. Sin embargo, no se obtiene una visión 3D del objeto.

Pipeline de transformación en OpenGL 3.3

En OpenGL moderno, el proceso de transformación de vértices se realiza mediante matrices. La ecuación fundamental es:

$$V_{clip\ position} = M_{projection} * M_{view} * M_{model} * Vértices$$

- **Vértices**: coordenadas originales del objeto.

- M_{model} (Model Matrix): transforma el objeto desde su espacio local al mundo (traslaciones, rotaciones, escalados).

- M_{view} (View Matrix): representa la cámara, es decir, desde dónde se observa la escena. Por defecto la cámara en el $0, 0, 0$ con dirección $0, 0, -1$ y orientación $0, 1, 0$. Para generar los diferentes planos bastará con girar la cámara, $gluLookAt(\ pos.x,\ pos.y,\ pos.z,\ dir.x,\ dir.y,\ dir.z,\ 0, 1, 0)$.

```
glm::mat4 view (1.0);
view = glm::lookAt(glm::vec3(x,y,z), glm::vec3(x,y,z), glm::vec3(x,y,z));
```

- $M_{projection}$ (Projection Matrix): define cómo se proyecta la escena.

  - **Ortográfica**: por defecto los valores son $glOrtho3D(-1,\ 1,\ 1,\ -1,\ -1,\ 1)$. Para generar las diferentes perspectivas multivista bastará con girar la cámara.

```
glm::mat4 proj =glm::ortho(-100.0f, 100.0f, -100.0f, 100.0f, 0.1f, 100.0f);
```

  - **Perspectiva**: $Aspect = (float)width/(float)height$

```
glm::mat4 proj = glm::perspective(45.0f, Aspect, 0.1f, 100.0f);
```

El resultado final, llamado $V_{clip\ position}$, se asigna a $gl_Position$ en el vertex shader.

- <u>Vertex Shader</u>

El vertex shader es el encargado de aplicar las transformaciones a cada vértice.

- **aPos**: posición del vértice de entrada.
- **model, view, projection**: matrices que se pasan como variables uniformes.
- **vec4(aPos, 1.0)**: convierte el vértice a coordenadas homogéneas.
- **gl_Position**: salida obligatoria del vertex shader.

- **glGetUniformLocation(shader, nombre)**: obtiene la posición de una variable uniforme en el shader.
- **glUniformMatrix4fv(location, count, transpose, value)**: location es un identificador de la variable, count es el número de matrices (normalmente

1), transpose determina si se transpone la matriz (normalmente GL_FALSE) y value es el puntero a los datos.

Vclip = Mprojection*Mview*Mmodel*Vertices

```
glm::mat4 model; // Se calcula
glm::mat4 view; // Se Calcula
glm::mat4 projection; // Se calcula

unsigned int modelLoc = glGetUniformLocation(IDShader, "model");
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));

unsigned int viewLoc = glGetUniformLocation(IDShader, "view");
glm::mat4 view = glm::lookAt(glm::vec3(x,y,z), glm::vec3(x,y,z), glm::vec3(x,y,z));
glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));

unsigned int projectionLoc = glGetUniformLocation(IDShader, "projection");
glm::mat4 projection =glm::ortho(-100.0f, 100.0f, -100.0f, 100.0f, 0.1f, 100.0f)
glUniformMatrix4fv(projectionLoc, 1, GL_FALSE, glm::value_ptr(projection));
```

```
#version 330 core
layout (location = 0) in vec3 aPos;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;
 void main() {
gl_Position = projection * view * model * vec4(aPos, 1.0);
}
```

22

Tipos de cámaras

- <u>Cámara telescopio</u>

Permite observar un objeto desde otro punto específico (por ejemplo, entre planetas). Usa: **glm::perspective** para proyección y **glm::lookAt** entre origen y destino.

```
void setupCamera() {
    glm::mat4 projection = glm::ortho(-1200.0f, 1200.0f, -1200.0f, 1200.0f, 1.0f, 3000.0f);

    glm::vec3 pos = glm::vec3(DISTANCIA_CAMARA * sin(alpha) * cos(beta), DISTANCIA_CAMARA * sin(beta), DISTANCIA_CAM
cos(alpha) * cos(beta));
    glm::mat4 view = glm::lookAt(pos, glm::vec3(0, 0, 0), glm::vec3(0, 1, 0));
   glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "projection"), 1, GL_FALSE, glm::value_ptr(projection));
   glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "view"), 1, GL_FALSE, glm::value_ptr(view));
}

void setupTelescope(ObjetoCeleste origen, ObjetoCeleste destino) {
    glm::mat4 projection = glm::perspective(glm::radians(45.0f), (float)SCR_WIDTH / SCR_HEIGHT, 0.1f,3000.0f);

glm::vec3 pOr = glm::vec3(origen.distancia * cos(origen.angulo_trans * GradosToRad), 0, origen.distancia *
sin(origen.angulo_trans * GradosToRad));
    glm::vec3 pDe = glm::vec3(destino.distancia * cos(destino.angulo_trans * GradosToRad), 0, destino.distancia *
sin(destino.angulo_trans * GradosToRad));
    glm::mat4 view = glm::lookAt(pOr, pDe, glm::vec3(0, 1, 0));
   glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "projection"), 1, GL_FALSE, glm::value_ptr(projection));
    glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "view"), 1, GL_FALSE, glm::value_ptr(view));
```

```
// --- Instancias de Planetas ---
ObjetoCeleste sol = { 0,0,  0, 1,  0, 100 };
ObjetoCeleste mercurio = { 200,  8,  0, 2,  0, 15 };
ObjetoCeleste tierra = { 500,  5,  0, 10, 0, 80 };
ObjetoCeleste luna = { 100,  15, 0, 20, 0, 20 };
ObjetoCeleste marte = { 800,  3,  0, 10, 0, 45 };
```

- <u>Cámara en primera persona</u>

Simula la vista desde un objeto (como un coche o jugador). La cámara está en la posición del objeto y mira hacia adelante según su orientación

```
void myPrimeraPersona(objeto coche_h) {

//Proyection Matriz
glm::mat4 projection;
projection = glm::mat4();
unsigned int proyectionLoc = glGetUniformLocation(shaderProgram, "projection");
projection = glm::perspective(glm::radians(45.0f), (float)SCR_WIDTH / (float)SCR_HEIGHT, 1.0f,
200.0f);
glUniformMatrix4fv(proyectionLoc, 1, GL_FALSE, glm::value_ptr(projection));


// View Matriz
glm::mat4 view;
view = glm::mat4();
unsigned int viewLoc = glGetUniformLocation(shaderProgram, "view");
view = glm::lookAt(glm::vec3(coche_h.px, coche_h.py + 6, coche_h.pz),
glm::vec3(coche_h.px + sin((coche_h.angulo_trans) * GradosToRadianes), 6, coche_h.pz +
cos((coche_h.angulo_trans) * GradosToRadianes)),
glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));


}
```

- <u>Cámara en tercera persona</u>

La cámara sigue al objeto desde atrás. Se posiciona detrás del objeto y mantiene una distancia fija mirando hacia el objeto.

```
void myTerceraPersona(objeto coche_h) {

// Proyection Matriz
glm::mat4 projection;
projection = glm::mat4();
unsigned int proyectionLoc = glGetUniformLocation(shaderProgram, "projection");//La
ponemos a uno
projection = glm::perspective(glm::radians(45.0f), (float)SCR_WIDTH / (float)SCR_HEIGHT,
1.0f, 200.0f);
glUniformMatrix4fv(proyectionLoc, 1, GL_FALSE, glm::value_ptr(projection));


// View Matriz
glm::mat4 view;
view = glm::mat4();
unsigned int viewLoc = glGetUniformLocation(shaderProgram, "view");
view = glm::lookAt(glm::vec3(coche_h.px - 20 * sin((coche_h.angulo_trans) *
GradosToRadianes), coche_h.py + 10, coche_h.pz - 20 * cos((coche_h.angulo_trans) *
GradosToRadianes)),
glm::vec3(coche_h.px + sin((coche_h.angulo_trans) * GradosToRadianes), 10, coche_h.pz +
cos((coche_h.angulo_trans) * GradosToRadianes)),
glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
}
```